

7주 차 보고서

(2026년 2월 8일 ~ 2026년 2월 14일)

작성자

박신우

이번 주 회의 내용

해당 주차 내 공식 회의가 없었습니다.

이번 주 한 일

2.9~2.10 좀비 따라옴 조건

플레이어가 좀비의 시야안에 있을 때만 플레이어를 따라오도록 하였다.

```
bool IsLineOfSightTo = LineOfSightTo(PlayerPawn);

if (IsLineOfSightTo)
{
    MoveToActor(PlayerPawn, 150.f);
    SetFocus(PlayerPawn);
}
else
{
    ClearFocus(EAIFocusPriority::Gameplay);
    StopMovement();
}
```

언리얼 엔진 내장 함수인 LineOfSightTo 함수를 이용하여 구현했는데, LineOfSightTo 함수의 정의를 보면 다음과 같다.

```
bool AAIController::LineOfSightTo(const AActor* Other, FVector ViewPoint, bool bAlternateChecks) const
{
    if (Other == nullptr)
    {
        return false;
    }

    if (ViewPoint.IsZero())
    {
        FRotator ViewRotation;
        GetActorEyesViewPoint(ViewPoint, ViewRotation);

        // if we still don't have a view point we simply fail
        if (ViewPoint.IsZero())
        {
            return false;
        }
    }

    FVector TargetLocation = Other->GetTargetLocation(GetPawn());

    FCollisionQueryParams CollisionParams(SCENE_QUERY_STAT(LineOfSight), true, this->GetPawn());
    CollisionParams.AddIgnoredActor(Other);

    bool bHit = GetWorld()->LineTraceTestByChannel(ViewPoint, TargetLocation, ECC_Visibility, CollisionParams);
    if (!bHit)
    {
        return true;
    }
}
```

아래 코드를 보면 AI의 눈에서 플레이어 위치로 라인 트레이스를 한다. 이때 ECC_Visibility는 시각적 가시성 채널을 사용하여, 벽이나 바위같이 불투명한 장애물을 판정 대상으로 삼는다고 한다.

```
bool bHit = GetWorld()->LineTraceTestByChannel(ViewPoint, TargetLocation, ECC_Visibility, CollisionParams);
if (!bHit)
{
    return true;
}
```

이때 바위나 벽에 부딪히면 bHit가 true로 반환되는데, false이면 바위나 벽이 없는것이므로 !bHit이면, 타겟이 시야안에 있다고 판단하여 true를 반환하는 구조이다.

이번 주 한 일

좀비가 잘 따라오다가 트럭 뒤에 숨으면 안 따라오는 모습을 볼 수 있었다.



프로그램을 실행하면서, 문득 그런 생각이 들었다.

저 코드를 보면, 그냥 AI의 눈에서 타겟의 위치로 직선을 그어서 판단하는데, 그러면 LineOfSightTo가 시야 안에 있는지 판단하는 게 아니라, 장애물이 있는지만 판단하는 게 아닌가 하는 생각이 들었다.

그래서 좀비 뒤로 접근해 봤는데, 바로 알아보고 쫓아오는 모습을 보였다. 좀비 시야랑 상관없이 그냥 플레이어와 ai사이에 장애물만 없으면 되는 거 같았다.

그래서 코드에서 아래와 같이 시야각 90도 안에 있는 경우에 알아보도록 하였다.

```
FVector Forward = GetPawn()->GetActorForwardVector();
FVector TargetDir = (PlayerPawn->GetActorLocation() - GetPawn()->GetActorLocation()).GetSafeNormal();

// 두 벡터의 내적 계산
float DotProduct = FVector::DotProduct(Forward, TargetDir);

// 시야각이 90도 이상인 경우 0.707이 90도에서의 내적값이므로, 0.7로 약간 여유를 둬
if (DotProduct < 0.7f)
{
    IsLineOfSightTo = false;
}
```

변경 후 등 뒤로 접근하니 안 쫓아오는 모습을 확인할 수 있었다.



이번 주 한 일



좀비 하체가 분리되어 기어다닐 때는 트럭 뒤에 숨어도 트럭 밑으로 기어서 다가오는 모습을 확인할 수 있었다.

좀비 눈에서 플레이어 위치로 직선을 긋는데, 기어다니면 좀비의 눈이 아래로 이동하여 트럭아래 틈으로 플레이어를 확인할 수 있어서 가능한 일 같았다.

2.11 트럭에 아이템 시각적 적재



아이템을 먹은 후 트럭에 상호작용기를 누르면, 트럭 위에 아이템의 모습이 보이도록 하였다.

적재된 아이템의 모습과 위치는 회의를 통해서 결정해야할 것 같다.

이번 주 한 일

2.12~2.13 좀비 공격

좀비가 공격하는 애니메이션이 없어서,
믹사모에서 애니메이션을 받아, 좀비 에셋에 리타게팅 해주었다.

좀비가 플레이어를 감지하고 추적하는 기능까지는 정상적으로 구현하였으나, 공격 기능 구현 과정에서 Tick 기반으로 공격 판정과 상태 전환을 처리하였을 때 공격 타이밍이 불안정하고 애니메이션과의 동기화 문제가 발생하여 의도한 대로 동작하지 않았다.

이에 따라 다음 주에 행동 트리를 이용하여 좀비의 Ai를 재설계할 예정입니다.

2.14 보고서 정리

보고서를 정리하면서 이번 주에 이것저것 알아보며 공부하느라,
진도를 빠르게 빼지 못했던 것 같다.
다음주에 더 열심히 해야겠다는 생각이 들었다.

다음 주 할 일

- 좀비 Ai 행동트리 이용해서 구현
- 트럭 아이템 적재 모습 좀 더 완성도 높게 만들기
- 윤진이 다텔하여 맵 메인에 병합시키기

특이사항